From Specifications to Implementation in the Gen-AI Era: Lessons from a Project-Based Software Engineering Course

YINGYING WANG, The University of British Columbia, Canada
MASIH BEIGI RIZI, The University of British Columbia, Canada
FATEMEH KHASHEI, The University of British Columbia, Canada
JULIA RUBIN, The University of British Columbia, Canada

By early 2025, AI code assistants had evolved into sophisticated collaborators capable of generating, explaining, reviewing, and modifying substantial portions of a software system. In February 2025, as we were delivering an upper-level undergraduate course on Software Engineering in our university, the term *vibe coding* emerged and quickly became popularized, referring to a practice in which developers describe a project or task in a prompt to an AI model that then generates code artifacts automatically.

In this paper, we first report on the course setup and our analysis of students' experience using AI assistants for developing open-ended software engineering projects in the Spring 2025 offering of the course. We then discuss our independent exploratory study that we conducted in Summer 2025, systematically investigating strategies for working with AI tools, specifically GitHub Copilot and Cursor, to implement a project of a real-world size and complexity, while also systematically evaluating the quality of the generated code. The goal of both studies is to better understand whether and how to teach Software Engineering in the AI era, i.e., which skills are essential for effective human–AI collaboration.

Our results show that students utilized AI tools in all stages of project development, to generate and refine artifacts and to learn unfamiliar concepts. While Generative AI tools simplified repetitive development tasks and helped quickly ramp up when working with unfamiliar frameworks and programming languages, students had mixed levels of satisfaction with using the tools, both as chat interfaces and as IDE-embedded solutions. We found no correlation between the degree of students' reliance on AI and their grades, hypothesizing that other factors, such as groups' knowledge and commitment, play a larger role in producing high-quality results.

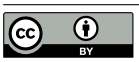
Our exploratory study shows that tools were still far from replacing software engineers or rendering software engineering education unnecessary. In fact, we observe that, in the new AI-empowered reality, skills such as requirements engineering, design, testing, and code review are becoming more relevant (and easier to teach) than ever. We conclude the paper with a set of suggestions for future offerings of this and similar Software Engineering courses, and for using AI in Software Engineering more broadly.

CCS Concepts: • **Software and its engineering → Software creation and management**; • **Applied computing → Education**.

Additional Key Words and Phrases: Software Engineering, AI-based development, vibe coding, human-AI interaction for software development

[*]The second and the third authors share equal contribution.

Authors' Contact Information: Yingying Wang, The University of British Columbia, Vancouver, Canada, wyingying@ece.ubc.ca; Masih Beigi Rizi, The University of British Columbia, Vancouver, Canada, masihbr@ece.ubc.ca; Fatemeh Khashei, The University of British Columbia, Vancouver, Canada, fatteme@ece.ubc.ca; Julia Rubin, The University of British Columbia, Vancouver, Canada, mjulia@ece.ubc.ca.



ACM 2994-970X/2026/7-ARTFSE064
https://doi.org/10.1145/3797092

ACM Reference Format:
Yingying Wang, Masih Beigi Rizi, Fatemeh Khashei, and Julia Rubin. 2026. From Specifications to Implementation in the Gen-AI Era: Lessons from a Project-Based Software Engineering Course. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE064 (July 2026), 24 pages. https://doi.org/10.1145/3797092

1 Introduction

As Generative AI (GenAI) has become the de facto norm in the software industry [1], with tools such as GitHub Copilot [11] and Cursor [6] being tightly integrated into modern development environments, we believe exposing students to the use of GenAI tools and teaching them how to use these tools responsibly and effectively have also become an essential part of upper-level software engineering education. To this end, since 2023 [27], we have allowed students to use GenAI tools throughout an elective upper-level project-based undergraduate course on Software Engineering, albeit in a controlled way, to fulfill the educational objectives of the course.

In this paper, we report on our experience with the course offering taught in the Spring 2025 term (January-April), which coincided with higher maturity and wider adoption of more sophisticated AI code assistants and the emergence of vibe coding [20] – a practice in which developers describe a project or task in a prompt to an AI model that then generates code artifacts automatically.

A distinct characteristic of our course, taken by students in their third to fifth years of studies, is an industrial-size student-defined software project, with an Android-based Kotlin mobile client and a Node.js-based TypeScript cloud server, which groups of four students complete in the scope of the course. Driven by the course-specific requirements, project features are defined by the students themselves. That is, each group produces a product of their choice, with some groups choosing to later deploy their projects in the Google Play Store. While this is not one of the course requirements, it attests to the type and complexity of the developed projects.

The course's incoming students have to complete at least two prerequisite programming courses, and about half have prior industry internship experience, either independently or through a co-op program. As such, the majority of the students in the course already have fundamental programming experience and take the course to focus on software engineering principles.

In this course offering, students were required to list the tools that they used for each milestone, to report on their impressions, and to critically analyze the advantages and disadvantages of the tools. They were asked to submit the results of their analysis together with the milestone artifacts. Students were not penalized for using AI tools, but failure to report was considered academic misconduct and incomplete reports were penalized by grading. We thus believe students' reporting is comprehensive and reliable, and use a mixed-methods approach to analyze artifacts produced by the course students and their corresponding impressions of GenAI.

Furthermore, in Summer 2025, we conducted an independent exploratory study, where we systematically investigated strategies for working with GenAI tools when implementing a large-scale project equivalent in size and complexity to those typically carried out by the students in the course. We further systematically evaluated the quality of the artifacts produced with each strategy, to draw conclusions on which strategies work well and which do not.

The main goal of our work is to advance understanding of whether and how GenAI-empowered development tools reshape tasks and priorities in Software Engineering and Software Engineering education, i.e., which skills are essential for effective human–AI collaboration. Specifically, we aim to (a) investigate the impact of GenAI on Software Engineering course offerings, (b) define educational goals and priorities for teaching Software Engineering in this new era of AI-empowered development, (c) inform our own and similar future courses on Software Engineering, and also (d) shed more light on ways GenAI can be utilized in Software Engineering more broadly. To this end, this paper aims to answer the following research questions:

RQ1: What was the students' experience of using GenAI tools in the scope of the Spring 2025 offering of the course? (Section 3)

RQ2: To what extent can GenAI augment software engineering processes and where does human expertise remain essential? (Section 4)

To answer **RQ1**, we performed both quantitative and qualitative analyses of student artifacts and reports. For the qualitative analysis, we used open and axial coding methods from grounded theory [65] to analyze student reports and identify a set of tasks where they employed Generative AI techniques, tools they used, and their impressions of using the tools for these tasks. We further explored whether there is a correlation between the degree of students' reliance on GenAI tools and the quality of the artifacts they produced (and, thus, their grades).

To answer **RQ2**, we conducted an exploratory study involving two software developers (with 4.5 and 2 years of industrial experience), who were graduate students at our university and who served as Teaching Assistants (TAs) of the studied offering of the course. We tasked them with the development of a project similar in size and complexity to those typically developed by the students in the course. We systematically explored a set of strategies for using GenAI tools when developing the project (ranging from "vibe coding" the entire project to a detailed set of well-verified tasks), analyzed the strengths and weaknesses of these strategies, and devised concrete suggestions to better guide students on the use of GenAI tools in the future offerings of this and similar courses.

Our results show that students used both chat-based tools, such as ChatGPT, Claude, and DeepSeek, and IDE-integrated tools, such as GitHub Copilot and Cursor. They used tools in all stages of project development – from requirements, through design, to implementation, testing, and code review – to generate and refine artifacts and also to explain concepts. To our surprise, ChatGPT was the most commonly used tool, even in the coding and testing phases, where IDE-integrated solutions were available. This is likely due to the high familiarity with the tool, as well as the low token limit in the free IDE-integrated solutions at the time of this course offering: both GitHub Copilot and Cursor were quite verbose and students quickly ran out of free credits, deciding to spread the work across multiple tools. Students had mixed opinions about the utility of the tools and we found no substantial correlation between students' reliance on GenAI tools and the quality of the artifacts they produced, hypothesizing that other factors, such as groups' knowledge, motivation, commitment, and effort play a larger role here.

The results of our exploratory study show that the GenAI tools we used were still unable to create fully-functional, high-quality, reliable, and secure code *without proper supervision of a highly-skilled human*. While these tools can simplify coding tasks, they are substantially more effective when professional knowledge is injected into prompting and result evaluations. Specifically, we observed that, for GenAI tools to function effectively and help the developers (instead of confusing them by generating a large amount of irrelevant, highly-bloated, and low-quality code), high *granularity* and *specificity* of tasks are necessary. Breaking development tasks into a logical sequence of smaller, well-defined steps requires a deep understanding of software engineering principles, such as requirements engineering, domain-driven design, and architectural decomposition. Furthermore, the quality of AI-generated code and tests often requires improvement, which makes skills in code review and efficient testing necessary.

We conclude that allowing students to use current GenAI tools in upper-level Software Engineering courses does not hinder their learning and that teaching proper Software Engineering practices is still essential. We hope that our analysis and suggestions will spark discussions and be informative for educators, AI tool builders, and software engineering practitioners. We believe that, with proper education and utilization, the future of AI-empowered Software Engineering

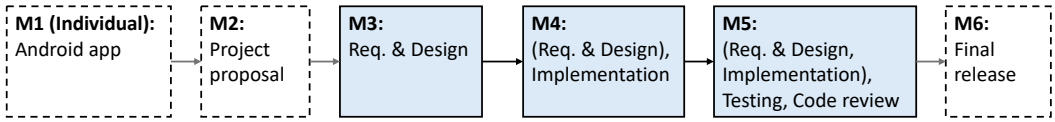


Fig. 1. Course Setup.

is promising, with humans focusing on the creative and logical aspects of development while AI handles mundane implementation details and programming language nuances.

Ethics. This work was conducted under the guidelines of the ethics board at our institution and received its approval. During the term, students were informed that their data, anonymized, might be used for research and teaching improvements. All authors of this paper were involved in the course in an instructional capacity. Unanonymized student data was not shared beyond the scope of the instructional team. To avoid any bias, the authors performed the data analysis reported in this paper after the grading for the course was completed and the grades were released to the students.

2 Course Setup

This elective upper-level undergraduate course focuses on teaching core Software Engineering principles required for building non-trivial software-intensive systems. Students can take the course starting from their third year of studies, after completing at least two prerequisite programming courses and several fundamental computer science courses, such as algorithms and data structures. Moreover, our university offers a co-op program during which students acquire professional, paid work experience through up to five co-op terms (resulting in a five-year-long undergraduate program). As such, many of the students are taking the course after completing at least one co-op internship in industry. Students in the course thus already have fundamental programming experience and take the course to learn software engineering principles and practices.

The course uses a term-long development project as the main learning vehicle. Working in groups of four, students develop a client-server software system of their choice, with an Android-based mobile client written in Kotlin and a Node.js-based cloud server written in TypeScript. Each project must contain three course-defined features (third-party authentication, integration with a third-party API, a live update functionality) and project-specific features defined by the students themselves.

This offering of the course had 69 students, who self-formed 18 groups. Around 60% of the course cohort was in their fourth or fifth year and around half of the course cohort had completed at least one co-op term.

The course consisted of six milestones, M1-M6, shown in Figure 1. The first milestone, M1, is an individual assignment, where every student followed a series of tutorials recorded by the teaching staff to build a simple Android/Kotlin application with a Node.js/TypeScript back-end deployed in the cloud, in two weeks' time. The goal of M1 was to get all students familiar with the technology stack they will be using for the rest of the course.

For M2, the students teamed up, scoped their corresponding projects, and submitted a proposal that describes their high-level project idea. Milestones M3-M5 are the main focus of this paper, where students iterated over different phases of the software development lifecycle. For M3, students defined the requirements and design of their project, which consisted of (a) a use case diagram and formal use case specification documents [44] for the main use cases of their projects, (b) a high-level design diagram showing by-domain decomposition of their code and its interactions with databases and external APIs, and (c) a detailed design of all interfaces exposed by the back-end to the front-end part of the project. For M4, they further refined the requirements and design, implemented the project, and submitted a working version with all use cases implemented. For

M5, they, again, refined the previous artifacts and also designed and implemented API-level and end-to-end tests for their project, including mocks for external APIs, to simulate failures. They also performed automated and manual code review. Finally, for M6, students were not expected to produce any new artifacts but rather address all feedback they received so far and refine their work: update requirements and design specifications, fix unresolved bugs, complete test suites, etc.

As the efficient and responsible use of GenAI tools has become a de facto business necessity in many areas of our lives, including software engineering, exploring how to use these tools effectively and understanding their strengths and limitations were also important educational objectives of this course offering. At the beginning of the course, we asked students to complete an entrance survey, to assess their prior experience using GenAI tools, including in courses taken earlier in the program, when allowed by the course syllabus. Throughout the course, students were allowed to use any GenAI tool of their choice (both chat systems and IDE-embedded solutions) as assistants in completing their work.

In this offering of the course, we provided no explicit guidelines on when and how to use the tools, as our goal was to better understand students' current level of knowledge, expertise, and experience and then use this information, together with results of our own exploratory study, to devise more explicit guidelines for future offerings of the course. To this end, we asked students to submit, for each milestone, a report documenting their use of the GenAI tools, in addition to the artifacts for the milestone. In the report, students were asked to describe which tools they employed, for what purpose, and to critically analyze the advantages and disadvantages of using the tools for that purpose. To avoid an easy "escape path", students who decided not to use any AI tools were asked to provide reasons for their decision and, when appropriate, examples of the tools' inadequacies.

Inadequate reporting was penalized by grades and using AI tools without proper reporting was considered academic misconduct. We thus believe students' reporting is reliable enough to use for analyzing students' experience and perspectives, towards suggesting further improvements to course structure and pedagogical methods.

Our full course syllabus and the list of reflection questions (which were communicated to students in the milestone description documents on Canvas) are also available in our online appendix [72].

3 RQ1: Students' Experience using GenAI

The entrance survey showed that all students had prior experience using GenAI and intended to use it in the context of their Software Engineering course assignments. To better understand students' experience of using GenAI tools in the requirements, design, implementation, and testing parts of the course (M3-M5), we posed the following research sub-questions:

RQ1.1: Which GenAI tools do students use in the scope of the course and for which tasks?

RQ1.2: What are the students' impressions when using these tools?

RQ1.3: Are the degree of GenAI reliance and the quality of the produced artifacts correlated?

We now discuss our methodology for answering these questions and present our results.

3.1 Methodology

Two authors of the paper independently read student reflections line by line, to collect the set of GenAI tools and tasks each group employed for milestones M3-M5. We also collected students' perceptions of the usefulness of GenAI tools in each task (positive, negative, or mixed feelings). When analyzing tasks and perceptions, we used open coding – a qualitative data analysis technique borrowed from grounded theory [65] – to identify concepts, i.e., key ideas contained in students' reflections. When looking for concepts, we searched for the best phrases that describe the tasks and

Table 1. GenAI Tools Used.

Tool	Milestone	M3 (Requirements & Design)	M4 (Implementation)	M5 (Testing)	Total
Chat Interface	ChatGPT	17	17	17	18
	Claude	1	2	5	5
	DeepSeek	1	3	4	5
IDE Integration	GitHub Copilot	2	8	6	8
	Cursor	0	2	2	2
Total		17	18	18	18

the students' opinions on the usefulness of the tools for each task. We further used axial coding [65] to group the tasks and perceptions into sub-categories and categories.

The two coders initially agreed on approximately 79% of decisions related to tasks and 86% of decisions related to perceptions. Disagreements, e.g., arising from different interpretations of student reflections, were first discussed between the two coders and resolved, when possible. Remaining disagreements (4% and 2%, respectively) were resolved in a joint discussion with all authors. The complete coding results are available in our online appendix [72].

To assess whether reliance on GenAI techniques increases or decreases students' grades, we looked for a correlation between the degree of students' reliance on GenAI and the quality of the artifacts they produced. To this end, we used students' self-reported data that estimated their perceived percentage of reliance on GenAI tools for the completion of the milestone. Then, for each milestone, we calculated the average percentage of reliance, e.g., 40%, and divided groups into two categories: groups below the average value were considered as 'Low Reliance' (denoted G_L) and groups at or above the average value – as 'High Reliance' (denoted G_H). We further calculated the average grade of all, G_H, and G_L groups, and compared them to each other, to investigate the differences in grades between the 'Low Reliance' and 'High Reliance' groups, if any.

3.2 Results

RQ1.1: Tools and Tasks. Our students used GenAI tools in two modes: via a chat interface or via an embedding into an Integrated Development Environment (IDE). Students report using ChatGPT, Claude, and DeepSeek directly via their chat interface. They used GitHub Copilot and Cursor within IDEs. While we do not have accurate information from all groups on the exact GenAI model they employed within their IDEs, we observe that many groups used Claude or picked the default mode provided by the tools, which automatically selects the best models for the task. Some students also mentioned that they experimented with multiple models and compared the results with each other, but did not find substantial differences between the different offerings.

Table 1 summarizes the number of groups that used each of these tools in milestones M3-M5. We further provide the total number of groups that used each tool in at least one of the milestones. As a group could use more than one tool in a milestone, the number of groups in each column does not add up to the total number of groups that used at least one AI tool for a milestone.

Specifically, for the requirements and design milestone (M3), one group decided to use no AI solution at all: «As a group, we worked efficiently to brainstorm ideas for the requirements and design of our project in labs and group meetings, so we did not need AI for our idea generation. Also, we understand the project and the app we are building better than AI does. As for the documentation, we have a clear template with guidance on the content and the format, so we did not see any point in using AI» (G_7).

Conversely, we were surprised to see that two of the groups used both a chat-based tool and an IDE-embedded solution, namely GitHub Copilot, when working on this milestone (which, on the

Table 2. Tasks Performed with GenAI.

Artifact / Mode	Documents	Requirements & Design	Implementation	Testing	Total
Generate	3	10	15	14	16
Refine	11	10	14	10	17
Explain Concepts	-	6	11	7	14
Total	12	17	18	18	18

surface, does not require any coding). Upon investigation, these groups utilized the tool to produce use case and sequence diagrams programmatically, using a diagramming language.

All groups used AI for the remaining milestones, with the majority of groups using chat-based vs. IDE-embedded solutions. We identified two reasons for such behavior: availability and familiarity. For the first reason, GenAI models within IDEs are generally quite verbose and expensive, causing the users to quickly run out of free credits after a relatively moderate use. As our students mentioned: «In Copilot, I usually just do some simple functions and easy-to-write things that are quite obvious to me and I can easily check. <...> With ChatGPT, I'd do stuff that is more complicated, such as debugging and figuring out what is happening.»

For the second reason, one of the students explained: «It has to do more with familiarity, since people were using ChatGPT before Copilot came around. I was very used to the interface. Now there is something new that you have to learn how to use». Some students also mentioned that the small size of the chat window in the IDE-embedded solutions is less convenient than the full-window chat interface, thus, again, gravitating to the most familiar ChatGPT.

Our analysis shows that a larger number of groups used GitHub Copilot compared to Cursor (8 vs. 2). We believe this was the case in our January 2025 course offering because (a) GitHub Copilot is an older, more familiar tool, with tighter integration into numerous development environments, including Android Studio [3], which our students had to use to implement a fully-functional Android front end, and VSCode [21], which is considered one of the most popular language-agnostic IDEs [18]. Also, (b) at the time of the course offering, students had access to GitHub education pack, which provided free access to GitHub Copilot, while educational offering for Cursor only became available in May 2025, after this course was over. We thus estimate that usage frequencies could vary in later offerings of the course.

Students used GenAI tools in three different modes: to *Generate*, *Refine*, and *Explain Concepts* related to artifacts. The artifacts associated with each mode, across milestones M3–M5, are listed as columns in Table 2: *Documents*, *Requirements and Design*, *Implementation*, and *Tests*. Each cell of the table shows the number of groups that reported a particular working mode (e.g., *Generate*) on a particular artifact type (e.g., *Tests*). The 'Total' column shows the total number of groups that applied GenAI in a particular working mode, across all artifacts. The 'Total' row shows the total number of groups that applied GenAI to help with a particular artifact type, across all working modes. As in Table 1, groups can use GenAI in multiple modes to work on multiple artifact types; as such, the number of groups in table rows and columns does not add up to the total of 18 groups.

Unsurprisingly, our analysis shows that most groups used GenAI tools to generate their implementation artifacts, as well as to refine these artifacts and generate tests. Refining documents and explaining implementation concepts (such as new programming languages and frameworks) were other popular usages of the tools. While we do not have complete information about which types of tools were used for which tasks, groups mentioned that they «used ChatGPT for brainstorming, initial code generation, refining design ideas, and helping with Espresso and Jest syntax, as well as GitHub Copilot for code suggestions and debugging assistance» (G_3).

Table 3. Students' Impressions.

Mode	Impression
Generate	+ Automating tedious tasks – Confusing/incorrect suggestions – Difficulties to understand/debug generated code – Outdated/deprecated libraries, incorrect dependencies – Do not precisely follow instructions – No global project context
Refine	+ Debugging issues and code review comments + Fixing grammar, clarity, and conciseness of writing + Feedback on requirements and design – No course-specific context – Repetitive unsatisfactory suggestions
Explain Concepts	+ Learning new languages, frameworks, etc. – Too generic / impractical solutions – Over-reliance

Interestingly, students' satisfaction is almost inversely proportional to the frequency of use, as we discuss next.

RQ1.2: Impressions. Table 3 summarizes students' impressions about the advantages and disadvantages of GenAI tools in each working mode.

1. *Generate*. Students were mostly dissatisfied with the quality of AI-generated artifacts. While most groups mentioned that generation saves time initially (e.g., «it allowed us to quickly generate code, especially in Kotlin, a language which all of us were unfamiliar with» (G_{11})), they observed the use of outdated / deprecated libraries or incorrect dependencies, redundant functions, incorrect code, and more: «When implementing the Firebase Messaging Client in Kotlin, the suggested code would prevent the project from building, and, on the back-end, ChatGPT would often write MongoDB queries that were inaccurate» (G_{17}).

Moreover, several groups mentioned that, even though AI is initially helpful to learn new programming languages and frameworks, «it can often [get us] into incorrect rabbit holes and waste a lot of time, if the developers do not know enough about the codebase and don't realize that the AI is completely off track» (G_8). Another group provided a concrete example where the lack of proper understanding of Android location handling mechanisms caused them a substantial delay in finalizing the M4 deliverable: «When we asked ChatGPT how we can get the user's current location in Kotlin, ChatGPT suggests that we can use fusedLocationClient.lastLocation. However, <...> the location will not be updated when we query the location later on, which is a critical bug since we need to use the user's real-time location to calculate the distance» (G_7).

A number of groups mentioned the difficulty of debugging and maintaining AI-generated code: «While the AI was quite good at explaining changes necessary for the modifications, we still found it difficult to maintain / develop on top of the generated starter code, especially for Kotlin, where we were somewhat unfamiliar with the framework's syntax» (G_{11}). Likewise, another group mentioned that «a lot of the code generated is not very modular <...> and the user cannot understand the rationale behind certain parts of the code» (G_{15}).

GenAI tools also did not precisely follow the user instructions: «it would attempt to unit test our files individually, despite specifically prompting the LLM to only write tests for endpoints» (G_{14}).

Finally, several groups mentioned that AI struggles to understand their project as a whole: «The system is getting too large for LLMs to understand, and it sometimes can't include the relevant parts of the codebase into the necessary context window» (G_5). Another group mentioned: «AI is disadvantageous as it lacks a broader perspective on the wider use cases and reasoning behind the

operation of the whole system. Some examples where this was difficult were in implementing the end-to-end tests, where the AI lacks fundamental reasoning on how the users will interact with the system and what problems they might face.» (G_{15})

On the positive side, students found GenAI useful to perform tedious tasks required for documentation, generation of comments, setting up test skeleton code, and other tasks with clearly defined outcomes, so that students «were able to focus more on creative problem-solving» (G_{18}).

2. *Refine*. Students' opinions about the use of GenAI for refining artifacts were mostly mixed. On the positive side, several groups found GenAI useful to refine their code and tests, especially in debugging issues and addressing code review comments: «AI technologies provided quick and relevant debugging suggestions, helping us identify potential issues faster than traditional debugging methods. This significantly reduced the time spent searching for solutions online and allowed us to focus on refining our code» (G_1). Our recent work found that one reason developers do not fix automated code review issues is that the warning message is not sufficiently informative [40]. Students thus used GenAI to help understand issues with unclear messages: «when we were unsure why a specific segment of code was generating a Codacy error, we put the code segment as well as the Codacy error into the AI and we were able to determine why the error was occurring, with further prompting.» (G_{15})

The majority of groups also appreciated GenAI's help in fixing the grammar, clarity, and conciseness of their writing. Some also mentioned GenAI tools providing useful feedback on the content of their requirements and design documents: «upon rephrasing for better understanding, we found there were some holes in our ideas, which we were able to catch early on. For instance, we forgot about the need for push notifications until we elaborated more on our recommendation algorithm, and found we had no framework or API to handle notifications to the user» (G_2).

At the same time, students mentioned the need for manual revisions to align with course-specific requirements: «The feedback provided by AI sometimes does not align with the specific course expectations» (G_8). Another group mentioned: «If the prompt given to it was too general or missing some information, it would provide an incorrect answer that would throw it off the right track, so we needed to understand fully what we were prompting it to do» (G_{16}).

Finally, in some cases, groups mentioned that GenAI repetitively produced unsatisfactory suggestions: «AI can get really caught up in one particular solution. We would tell AI our problem and it kept suggesting the same thing even if we told it that was not the fix» (G_{13}).

3. *Explain Concepts*. Students were mostly satisfied with GenAI for explaining concepts, especially when learning new programming languages, frameworks (e.g., Espresso [9] and Jest [14]), technology stacks, algorithms, etc., as GenAI saves search time and presents relevant information in a condensed manner. For example, one group valued AI's ability to «provide trade-offs for using different frameworks/technologies for the same task, so it was easy for us to compare and pick the optimal tool for tackling a requirement» (G_{14}). Another group mentioned: «While it is possible to learn the math for route generation by reading online resources, finding a specific answer to something you don't understand can be difficult. ChatGPT makes it easier to get targeted explanations» (G_4). Students also found GenAI «especially useful because instructors, TAs, and peers have limited availability» (G_8).

On the negative side, students were concerned with results being inconsistent with conventions used in class or too impractical / expensive for the scope of the course, e.g., «[GenAI] provided suggestions that are either too expensive or overkill, such as suggesting the use of PineCone for our vector database when ChromaDB is a way cheaper alternative» (G_{14}).

Students were also largely aware of the risk of over-reliance on AI. For example, one group mentioned that «It provides the solution to our questions right away, so we don't have to dig down

Table 4. Reliance on AI vs. Grades.

	M3	M4	M5
Grade range	42-94	60-100	68-94
Average grade (all groups)	71.7	86.0	80.4
Average AI reliance (%)	26.7	40.9	47
High:Low reliance groups (G_H:G_L)	5:13	8:10	9:9
Average grade, G_H	75.7	88.1	79.6
Average grade, G_L	70.2	84.3	81.2

and perform the research by ourselves to verify the solutions work, which blocks the thinking process. Sometimes it provides the wrong result and we accept those results» (G_2).

RQ1.3: AI Reliance vs. Grades. Table 4 shows the correlation between the students' AI reliance degree and their grades, separately for each milestone. For M3 (Requirements and Design), students report a relatively low level of reliance on GenAI (average of 26.7%), with only five groups (denoted by G_H) having average or above reliance. The average milestone grade for these groups was 75.7, around five points higher than the average grade of 70.2 for low-reliance groups (denoted by G_L) and the overall average grade of 71.7 for the milestone. For comparison, the grades for this milestone ranged between 42 and 94, indicating sufficient variation overall.

While the overall reliance on GenAI techniques grew in the M4 and M5 milestones, the difference between average grades for G_H and G_L groups got smaller: 88.1 vs. 84.3 for M4. The trend is reversed in M5, where high-reliance groups actually scored lower than low-reliance groups: 79.6 vs. 81.2 for G_H and G_L, respectively, both close to the overall average of 80.4 for this milestone.

We further ran a Spearman correlation test [64] to examine the relationship between grades and GenAI reliance, and a Mann-Whitney U test [52] to compare grades for G_H vs. G_L groups, as both tests are appropriate for data of our size and distribution. We found no statistically significant correlation between students' GenAI reliance and their grades and no statistically significant difference in grades between G_H and G_L groups. While these statistical results should be interpreted with caution due to the small sample size (i.e., the number of groups in this study), we still believe that the observed grade differences are not attributable to the degree of reliance on GenAI tools but rather to other factors, such as groups' knowledge, commitment, and effort.

Answer to RQ1: Students used both chat-based and IDE-embedded GenAI tools for generating and refining artifacts (such as requirements, design, implementation, tests, and documentation) and for explaining concepts (especially when debugging or learning new programming languages, frameworks, and technology stacks). Students were mostly satisfied with GenAI tools for explaining concepts, mostly dissatisfied with the tools for generating artifacts, and had mixed opinions about the refinement process. Yet, we observed no substantial correlation between the students' grades and their degree of reliance on GenAI.

4 RQ2: Learning with GenAI

To better understand the capabilities of the tools, the skills necessary to use them effectively, and to assess the need for refining the course curriculum in the presence of GenAI tools, we decided to conduct our own independent controlled experiment, which we discuss next. The goal of this experiment was to systematically explore a set of strategies for using GenAI tools (ranging from "vibe coding" the entire project to a detailed set of well-verified tasks), analyze the strengths and weaknesses of these strategies, and devise concrete suggestions to better guide students on the use of GenAI tools in the future offerings of this and similar courses.

4.1 Methodology

Team and Task. For our experiment, we recruited two of the course's Teaching Assistants (TAs), who also had 4.5 and 2 years of industrial software development experience, respectively, and who subsequently became co-authors of this paper. We refer to them as expert developers throughout the paper. We tasked them with developing a project similar in size and scope to projects developed by students in the course. Specifically, inspired by several similar student projects, we decided to implement a *MovieSwipe* application that targets groups of friends, families, or coworkers who want to watch movies together but often struggle to find a movie that satisfies everyone's diverse preferences. The developers kept observational notes, documenting their experience throughout all steps of this study; we analyzed these notes to draw conclusions from the experiment.

Project Specificity. We experimented with three levels of specificity in the project details provided to GenAI: a high-level app description, a list of main app features, and a list of formal use case specifications [44] of all app use cases. These are described below and represent different levels of requirements engineering knowledge a developer might need to effectively produce useful and accurate software.

1. High-level App Description: The name of the app is MovieSwipe. In the app, a user can create a group and invite other users to join the group via an invitation code. The group owner can also delete the group. Upon joining a group, users are asked to specify what genres of movies they like (comedy, horror, action, etc.). The group owner can then start a voting session and the app shows 10 movie recommendations that group members can swipe right for "interested" or left for "not interested". MovieSwipe's smart recommendation system looks at everyone's choices and finds the best match that most people will be happy with. When voting ends, the app shows the winning movie with its details to everyone in the group.

We intend the app to use Google authentication [4], the TMDB [17] external API to retrieve a set of movies by genre, Firebase Cloud Messaging [10] to notify group members when the owner starts and ends the voting session via push notification, and Socket.IO [16] to automatically update the user interface in several places (live updates), e.g., when a member joins or leaves a group.

2. App Feature List: MovieSwipe has four main app-specific features and a more "generic" Google authentication feature specified in the course project requirements.

(1) *Manage Authentication*: A user must sign up for the app on the first use and then sign in for each session using Google Authentication Service. An authenticated user can sign out or delete their account altogether.
(2) *Manage Groups*: A user can create a group and become its owner. When creating a group, the group owner provides the group name and specifies their movie genre preferences. The list of genres is retrieved from the TMDB API. After creating the group, the group owner can view group details such as the group name, a unique invitation code for member recruitment, and their role in the group ("owner"). The group owner can also delete groups they own. Any user can view a list of active groups they own or have joined and details about these groups. Once a user joins or leaves a group, the list of members is automatically updated on every open screen.
(3) *Manage Group Membership*: A user can join a group using the invitation code shared by the group owner and become a group member. When joining, the user must specify their movie genre preferences, selecting from the list of genres specified for the group by its owner. The user can leave the group at any time, if they wish.
(4) *Manage Voting Session*: The group owner can start a voting session, triggering the app to generate a list of 10 recommended movies, based on all members' genre preferences.

Once the list is ready, the app sends, via Firebase Cloud Messaging, real-time notifications to all members announcing the start of the voting session. Group members can tap the notification to be redirected to the app to join the voting session.
The group owner can end the voting session, triggering the app to analyze votes and determine the winning movie. The app then sends push notifications to all group members via Firebase Cloud Messaging. When group members tap the notification, the app displays the winning movie, along with the details of the movie.

(5) *Vote for Movie*: In the voting session, each group member indicates their movie preferences by swiping right for "interested" or left for "not interested" on each movie card. Movie cards display movie metadata. Users can exit the voting session and have their voting results recorded, then resume the voting session if they exited before ranking all movies.

3. Formal Use Case Specifications: We further broke the main app features into 15 detailed use cases, such as *Create Group*, *Delete Group*, *Join Group*, *Vote for Movies*, etc. We devised a formal use case specification for each such use case, describing how a user (an "actor") interacts with a system to achieve the specific goal and outlining the main triggers, success path, and alternative failure paths. The full version of our formal use case specifications is available online [72].

Design Granularity. We further experimented with different levels of granularity when decomposing development tasks into smaller, more manageable pieces [32, 55], starting from (1) a full project at once, and gradually reducing the level of granularity to (2) by-frontend-backend, (3) by-feature, (4) by-use-case, and (5) by-design-component generation. In particular, at the lowest level of granularity, by-design-component, we split the development of each use case by tasks aligned with the system's architecture layers, i.e., separately and sequentially generating database handlers, middleware, controllers, routes, APIs in the back-end and UI screens, ViewModel logic, data sources, repositories, and back-end API integration in the front-end. This approach relies on the developers' knowledge and ability to generate the by-domain and by-architectural-layer design of the system [60] and further use this design to enable independent work on each software component. The full description of the design patterns and tasks that we used is available online [72].

Additional Configurations. Based on our initial experiments (spoiler alert: "brute-force" project generation did not work that well), we further provided the AI tools with additional configurations and guidelines, such as code quality, security, performance, and testing guidelines taken from online repositories. The reasons for introducing these guidelines and the outcomes of using them will be further discussed in the results section, after the appropriate context is provided.

Assessment Criteria. We assessed each code generation experiment in two dimensions: the time investment necessary to complete the work (in hours) and the quality of the produced project. For project quality, we utilized the criteria used in the course for grading student-submitted artifacts, which we adopted from the ISO/IEC 25010:2023 standard [13] and adapted to fit the course format:

(1) *Project Structure* assesses the general project scaffolding: organization of the project repository, structural clarity in the definition of various components, and the overall setup of the project.
(2) *Completeness* assesses whether the project contains all described functionality and meets core course requirements, such as live updates and the use of external APIs.
(3) *Correctness* assesses whether the code compiles, can be deployed, works as expected, with inputs that are validated, errors are handled, etc.
(4) *Code Quality – Maintainability* assesses whether the code has clear documentation and is free of maintainability issues such as convention misuse (e.g., for REST), deprecated APIs and libraries, complex and unnecessary code, magic numbers, etc.

(5) *Code Quality – Security* assesses code integrity (e.g., API protection) and confidentiality (e.g., no secrets are stored in code, data is encrypted in motion, etc.).
(6) *Code Quality – Performance* assesses algorithmic, resource, and network efficiency of the code.

In our evaluation, we experimented with different combinations of specificity and granularity, producing a number of *scenarios*, which we describe in the following section. For each scenario, we ran the tools three times, selecting for further analysis a version that appeared most complete and correct. Then, two authors of the paper assessed code artifacts for each evaluation criterion and cross-validated each other's findings. They resolved any disagreements (around 5%) in a joint discussion and involved a third author of the paper, when necessary.

GenAI Tools. We started by using GenAI tools utilized by the students for the implementation tasks, namely GitHub Copilot and Cursor. After some experimentation, we decided to use Cursor as our primary tool for the study, given its more advanced recent support for interactive, code-centric development workflows, recent popularity, and availability of an academic free plan. Within Cursor, we picked the claude-sonnet-4 model [12] as it was both recommended in the Cursor model selection guide [8] for a wide range of code-related tasks and was ranked high by developers in the Stack Overflow 2025 Developer Survey [1] at the time of our study.

4.2 Results

Figure 2 shows the possible granularity and specificity combinations. We shade in grey unfeasible combinations, e.g., generating code feature-by-feature without providing a feature list. Due to space limitations, we focus on the most prominent of the remaining combinations, which we number sequentially and discuss next.

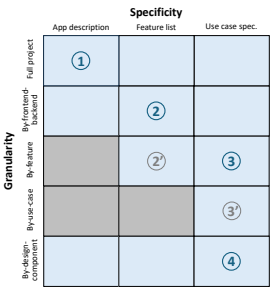


Fig. 2. Scenarios.

Scenario 1. We started our assessment from the most coarse-grained scenario w.r.t. both specificity and granularity: we asked the GenAI tool to generate the entire project based on its high-level description. This scenario simulates the "no software engineering knowledge is needed" direction, taking the idea of vibe coding [20] to its extreme. We invested around one hour of effort crafting a prompt that (a) specifies the course and project requirements, directly copied from the course syllabus, and (b) provides a high-level app description, as specified in Section 4.1.

As probably anticipated, the produced project did not meet our quality expectations. Figure 3 outlines our assessment of the knowledge required, the quality of the produced outcome, and the time invested in this and all subsequent scenarios we executed. As the figure shows, the produced project suffered from numerous issues: while the structure of the back-end code was mostly reasonable, the front-end code was not structured properly (probably due to the lack of coherent examples in the training datasets). Moreover, only a subset of the requested features was present and, at the same time, there were many unintended and unnecessary behaviors implemented, making the code extremely hard to understand (similar to the experience mentioned by our students, who had difficulties reading AI-generated code). In addition, there were numerous configuration, dependency, compilation, and build errors. The quality of the code was extremely low, with missing error handling, validation of request parameters and responses from third-party libraries, incorrect typing, hard-coded secrets, use of libraries with known critical security vulnerabilities, and more.

Figure 4 outlines the main issues in AI-generated code that we observed, which we further classified by the issue type.

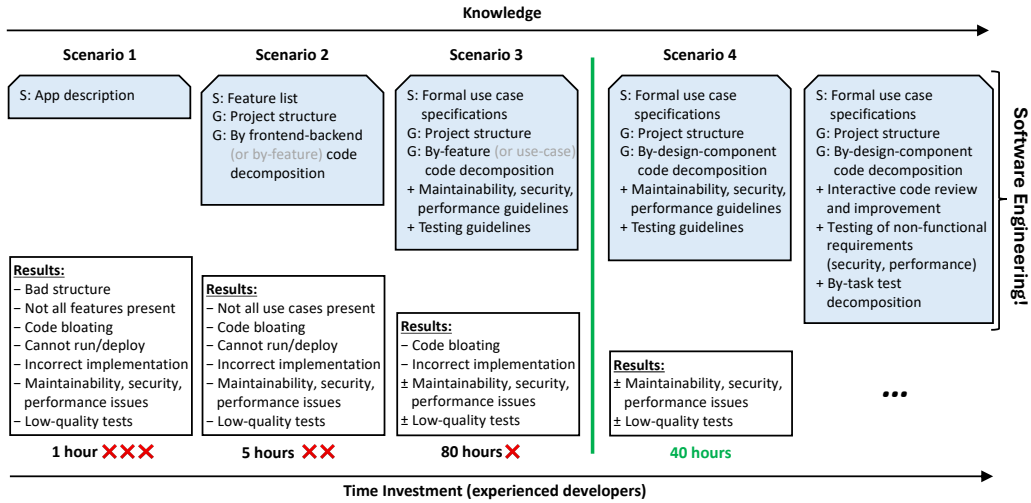


Fig. 3. Assessment of AI-generated Code.

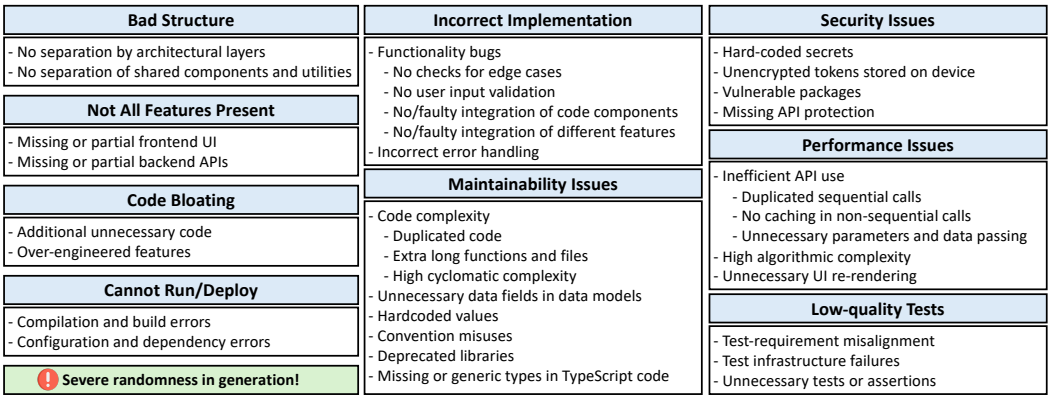


Fig. 4. Issues of AI-generated Code.

As discussed in Section 4.1, we repeated each scenario multiple times, picking the best result for further analysis. In this process, we observed a high variation between runs, which we attribute to the high randomness of the generation at such a coarsely defined task. While the effort that went into generating the code in this scenario was extremely low, the produced outcomes proved to be unworkable: after an inspection, all authors of the paper agreed it would be easier to implement the project from scratch than to further identify and fix all issues in this project.

Scenario 2. Here, we increased both the specificity and granularity of our instructions: we provided well-defined guidelines on how to structure the front-end and back-end code, following MVVM-like Android's official guidelines and the MVC architecture for the front-end and back-end, respectively. We also provided the list of application features, as specified in Section 4.1. Finally, we instructed the tool to first generate the complete back-end code, together with its corresponding API documentation. Then, we instructed the tool to generate the front-end code, providing the API documentation as input. Such practice of separating the front-end and back-end development is common in industry, to maximize expertise in particular technical stacks. In fact, one of our expert developers worked on the front-end and the other – on the back-end parts of the project.

They spent around five hours collectively crafting the prompts for this scenario, all of which are available in our online appendix [72].

We observed that the information we had input into the process, such as the desired code structure and list of features, helped the tool to focus better and more functionality was implemented. Yet, not all use case scenarios for each feature were implemented, e.g., the delete account use case was missing in the Manage Authentication feature. In a similar scenario (Scenario 2', marked in grey), we further extended the granularity to generate code feature-by-feature and, internally, by front-end and back-end. To our surprise, even under this setup, use cases were incomplete, e.g., all group members could start and stop the voting session, despite explicitly specifying that it should be done by group owners only.

In both cases, the generated app did not build successfully due to missing libraries and could not be deployed. While some quality issues we observed in Scenario 1 did not occur in this generation, new issues that did not occur before appeared. In a nutshell, the front-end part assumed data in the back-end responses that does not exist according to the API specification, the back-end missed error handling and input validation, had unnecessary APIs, unused imports, poor typing, hard-coded environment values and external API URLs, and used magic numbers in movie ranking calculations. In addition, it included duplicate code for checking if the user is authenticated: in the middleware and then, again, in the controller.

Finally, the tool generated tests, but these tests were sporadic and partial. As with Scenario 1, we observed high variance between different executions of this scenario as well.

Scenario 3. To ensure the generated code includes the exact functionality required for the project, not more or less, we further increased the specificity and used a set of formal use case specifications to describe each application feature, as specified in Section 4.1. In an attempt to further focus the generation by reducing the scope of the task at hand, we also further increased granularity, prompting the tool to generate code feature-by-feature (and also by use case in Scenario 3'). Moreover, inspired by the online vibe coding community, we extracted relevant code quality, security, and performance rules for the Cursor directory [2, 7, 15, 19, 22] and adapted them to the scope of our project. We further provided testing guidelines relevant for our project.

With this scenario, we simulate a developer with deep knowledge in requirements engineering, and awareness of the importance of architectural, code quality, security, performance, and testing guidelines (that can be collected online). It took us around 25 hours of work to define detailed use case specifications; collect, filter, and set up Cursor rules; and devise by-feature code generation prompts. All these artifacts are available in our online appendix [72].

After this investment, even though the generated code more accurately represented our intention (thanks to the detailed use case specifications), it still contained unused code and UI screens, duplicated components, and unreachable code, contributing to the code bloat and difficulties in understanding and further troubleshooting issues. We further observed incorrect functionality (e.g., failing sign-out); code quality, security, and performance issues, albeit a slight improvement thanks to the provided rules, still persisted. Finally, despite the provided testing guidelines, tests were mostly incomplete, e.g., some UI tests checked that UI elements existed rather than performing end-to-end testing as requested.

The expert developers driving the experiment checked the code at the end of each generation task (feature or use case), to make sure the code compiled, ran, and was functional as expected. When issues were found, they prompted the tool to resolve them for up to three times, before resolving the issue manually in order to move on with the process. Overall, they spent almost two weeks (around 55 hours) trying to identify and fix the main issues in this generation; they ultimately found this process excessively long and not that enjoyable (claiming it would be easier to write the code themselves).

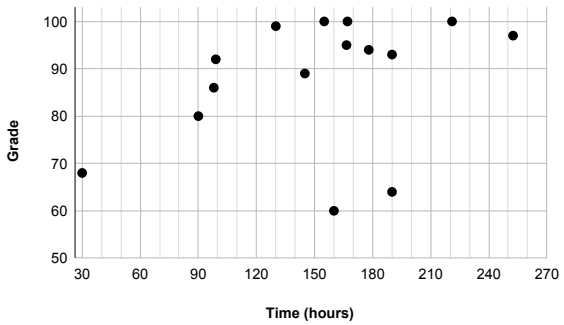


Fig. 5. Students' Reported Time vs. Grades in Implementation Milestone (M4).

Scenario 4. To further reduce the overhead in inspecting large chunks of low-quality generated code, in this scenario, we supplemented the tool input with a detailed design diagram and increased the granularity to a by-design-component generation, splitting the development of each use case into tasks aligned with the system's architecture layers, as specified in our online appendix [72]. As with Scenario 3, our expert developers inspected the code at the end of each generation task, prompted up to three times to fix it, if issues were found, and made sure to manually fix the code when needed, before proceeding to the next task.

With such a detailed logical split, the generated code was more precise and often functional, meeting our expectations. Overall, the developers spent around 15 hours producing a working version of the app, in addition to the 25 hours already spent on defining use case specifications and Cursor rules. This produced an app aligned with the expectations of the version students had to produce for their M4 milestone (Implementation) without yet ensuring proper code quality and app testing.

Indeed, we observed that the code still exhibited occasional code quality, security, and performance issues. However, the issues were easier to spot and fix due to a smaller scope of the generation tasks. We believe most of the additional necessary effort would be spent on refining and fixing tests, which will rely on a similar process we followed when producing code: fine-grained instructions for test generation and manual validation/fix after each generation step.

Effort Estimation. To put the effort estimates in perspective, Figure 5 shows the relationship between the groups' self-reported time investment in completing the coding part of the project (M4) and their corresponding grades.

Even though no direct time comparison is possible, as students are still in their learning phase, we expect students to invest two to three times longer in developing the project compared with the developers employed in this experiment. With proper use of AI techniques, this could be 80-120 hours. Improper use could lead to lower grades or unreasonable time investment of up to 160-240 hours, consistent with the numbers reported by students.

We thus believe educating students on how to properly utilize GenAI techniques can increase their efficiency, effectiveness, job prospects, and satisfaction.

Answer to RQ2: While AI can provide valuable assistance, software engineering knowledge and best practices are still essential to produce high-quality outcomes. This includes requirements engineering, design, detailed decomposition of generation tasks, code review, iterative development principles, and more. Students (and other software engineering professionals) need proper education on how to use GenAI tools responsibly, so that these tools assist rather than hinder their work.

5 Threats to Validity

External validity pertains to conditions that limit the generalization of our findings. As in many other exploratory studies and experience reports, our work is inductive in nature and thus might not generalize beyond the subjects that we studied. Yet, we believe that the experience of a total of 18 groups of students working on a diverse set of project ideas (that came from the students themselves), as well as our own exploratory study on the topic, would be valuable for other large-scale software engineering courses. Furthermore, we make all our artifacts available to encourage replication and reproducibility [72].

Internal validity is concerned with the causality of the results. For student data analysis, we may have misinterpreted students' answers, introducing bias to the analysis. Likewise, for our exploratory study, we generated all prompts and manually assessed the code produced by GenAI tools, which could also inject unintended bias. To mitigate these threats, all generated concepts (requirements, design, prompts, guidelines) were reviewed by all authors of this paper. Data analysis was performed independently by two authors of the work and all disagreements were, again, discussed and resolved by involving additional authors.

Furthermore, GenAI tools often exhibit non-deterministic behaviors. This means that the same prompt can yield different outputs across runs. This randomness may affect the reproducibility of our results. To improve transparency and mitigate this threat, we make our study methodology, prompts we used in each scenario, tool configurations, and the generated code publicly available [72].

Construct validity concerns whether the study builds up the correct operational measures for the concept being studied. Our student data analysis relies on students' self-reported data, which raises a concern that students could have been dishonest with their answers. However, we believe the probability of dishonest answers is low: students were not penalized or rewarded for using GenAI tools and were rather penalized by grades for incomplete reporting.

Confounding factors such as students' prior experience, financial resources, and tool familiarity, could limit the strength of the comparison drawn between high- and low-reliance groups. However, as we did not provide any restrictions or guidelines on which tools to use, we believe our work achieves our goal of characterizing current practices and experiences with GenAI tools in the scope of a software engineering course. We also confirmed many of the students' experiences with our own observations in our exploratory study.

6 Discussion, Implications, and Future Work

Software Engineering Education. Based on our experience, we not only believe that Software Engineering education remains absolutely necessary, but we also find that teaching it has become easier and more engaging: in the past, Software Engineering instructors often struggled to motivate students to learn topics such as requirements engineering and design, because their benefits – efficient communication, teamwork, and long-term project maintainability – were abstract and too futuristic for students to grasp. With the integration of GenAI tools, these concepts become immediately tangible: students can see how well-structured requirements and designs directly improve the performance of AI-assisted development, making the relevance of core Software Engineering principles clearer and more compelling. In a sense, AI now allows the instructors to demonstrate the immediate relevance of concepts taught.

Likewise, topics such as code review, version control, and testing have become more essential than ever, as students must review and validate AI-generated code, maintain the latest working version, and run regression tests consistently. We believe Software Engineering course curricula should be augmented and restructured to emphasize these AI-induced needs, which are largely addressed by the "classical" Software Engineering concepts.

We also plan to introduce students in the next offerings of this course to the outcomes of our study, from which we derive strategies and best practices of using GenAI during the course project development: clearly specifying requirements and design of the target product, separating code generation into small, well-controlled tasks (typically, aligned with product design) to reduce code bloat and support more efficient code validation, thoroughly validating the generated artifacts and manually adjusting when necessary (to avoid following AI-induced rabbit holes). We intend to compare the quality of the students' generated artifacts, their time investment, and their impressions to those of students in the current course offerings.

We further plan to devise guidelines extending to the testing and code review stages, i.e., exploring concrete strategies for breaking test generation processes into tangible steps. Besides our own experiments, we are also considering making this task a part of students' testing assignment (M5), encouraging them to experiment independently and apply concepts learned in code generation for efficient generation of tests.

For the code review phase, our current findings clearly demonstrate the need to review AI-generated code, and we intend to expose students to these examples to increase their awareness and understanding of differences between working code/prototype and high-quality, secure, performant, and production-ready code. In fact, in all our course offerings, we ask students to use a select set of static checkers, e.g., provided by Codacy [5], to improve the quality of their code. We applied Codacy to projects generated with GenAI assistance in our experiments reported in this paper and observed a large number of reported issues, some of which are quite major.

Utilizing AI to Assess Open-ended Student Projects. Another promising direction is to evaluate the use of AI to support instructors in code review and grading, when each student group works on their own independent project. In such settings, where no single "one-size-fits-all" solution exists for requirements, design, and other project-specific considerations, auto-grading is largely impossible and human feedback and oversight remain a critical yet expensive resource. Using data that we collected on quality issues in AI-generated code (e.g., Figure 4), we intend to devise AI-based approaches to provide initial assessments of the quality of student-submitted artifacts, which could substantially reduce the burden on teaching staff. While work on automated AI-assisted code review is ongoing, preliminary evidence suggests that it is still far from providing reliable, context-aware evaluations, as discussed in the next section.

Moreover, effective and holistic assessment of artifacts beyond code, i.e., the consistency between requirements, design, implementation, code, and tests, is also needed. One promising direction here could be the injection of traceability information during the artifact generation process, as was explored in the past for the model-driven development paradigm [24], and using this information to identify relationships between artifacts to guide the assessment.

Learning Outcomes. A recurring concern expressed by students is whether reliance on GenAI tools could undermine their critical thinking capabilities. While our results suggest that careful task decomposition and structured use of AI can rather strengthen software engineering type of reasoning, our experiments using the tools were conducted by software engineers trained before GenAI tools became mainstream. We believe future studies are needed to investigate whether students who acquire software development skills in the GenAI era develop critical thinking differently from those trained in more traditional settings. Future work could also investigate how working with GenAI affects students' ability to reason about trade-offs, evaluate alternative designs, and independently validate results. Based on this analysis, we intend to further augment the course curriculum, focusing on skills students need to become successful in the new world of human-AI integrated software development.

7 Related Work

GenAI in Software Engineering Education. The majority of work in software engineering education focuses on introductory programming courses [58] or isolated software engineering tasks. For examples of the latter, Choudhuri et al. [29] systematically studied the effectiveness of ChatGPT in assisting students with debugging, fixing code smells, and creating a git branch, granting only the experimental group access to ChatGPT. The authors found no statistical differences in participants' productivity or self-efficacy when using ChatGPT as compared to traditional resources. Likewise, our own prior work [27] focused on the requirements engineering phase, designing two distinct processes for performing the task with and without GenAI assistance, and found no statistically significant difference between the processes.

Others used surveys, observational studies, as well as analysis of students' artifacts and reflections to investigate students' usage and perceptions of GenAI tools [25, 30, 42, 59]. These works found that students use GenAI for a wide range of tasks in requirements engineering, code generation, testing, and documentation, including the generation of code skeletons and individual snippets that they later integrate into a larger codebase, helping with syntax and debugging tasks, refining code to comply with standards and conventions, as well as learning new concepts and refining understanding of known ones. Students perceived GenAI tools as beneficial for learning and productivity improvement while expressing concerns about the low familiarity with AI-generated code, errors in the generated code, over-reliance on AI, and general concerns about the future of the software engineering job market. Our findings are consistent with these studies. However, we do not rely solely on students' reports and artifacts but also conduct an independent study to compare different GenAI usage modes and provide concrete suggestions for efficient human-AI collaboration and task decomposition.

Salomon et al. [62] performed a large-scale study as part of a software engineering course, where students worked in pairs to build a data store of university courses and teaching spaces, iterating over three versions of the system. The authors observed that students spent less time on requirements and design and tended to jump into implementation earlier, often using GenAI to produce initial templates or starting points. Furthermore, students used GenAI not only to generate code but also to validate work retrospectively, such as checking alignment with requirements and design after implementation had begun. Students often turned to GenAI as their first point of contact, even before consulting teammates, which reduced direct interpersonal collaboration. We see our works as complementary: while our study did not explore the teamwork aspect in depth, the different nature of our course setup caused requirements and design phases to become more prominent, as groups worked on their own project ideas vs. the same predefined project. Both studies suggest the need for new pedagogical strategies that address both individual tool use and collaborative practices in GenAI-augmented teams.

Alpizar-Chacon and Keuning [25] and Shah et al. [63] investigated characteristics of students' prompts, showing that students often prompt once to generate the entire feature, followed by additional prompts to debug or regenerate code. Our study provided guidelines to improve prompting, e.g., via by-use-case and by-design-component task decomposition, focusing on substantially larger-scale projects.

Lee et al. [45] surveyed 319 knowledge workers and found higher confidence in GenAI is associated with less critical thinking, while higher self-confidence is associated with more critical thinking. The authors also found that GenAI shifts the nature of critical thinking toward information verification, response integration, and task stewardship. These findings could be inspirational for developing future educational agendas.

GenAI in Software Development. Industrial opinions on the effectiveness of GenAI in software engineering tasks are mixed. According to the 2025 Stack Overflow developer survey [1], 80% of the 49,000 survey participants use GenAI tools in their workflows. However, trust in the accuracy of GenAI dropped from 40% in previous years to just 29% in 2025. The top frustration (voted by 45% of respondents) is that GenAI generates solutions that are "almost right but not quite", making debugging more time-consuming. A recent ACM Tech Brief discussed similar benefits and risks of vibe coding [37] and METR Research found that GenAI slows experienced software developers (recruited from 16 open-source repositories) down [28].

Similar to our findings, prior work also reported that code generated by GenAI tools has various issues related to correctness [41, 47, 49, 50, 53, 54, 73], security [23, 26, 35, 50, 56, 69], performance [23, 46, 53], complexity [50, 54], and maintainability [23, 49]. Several authors also shared their own and other developers' experiences of using GenAI tools. They emphasize the need to provide GenAI tools with functional requirements and domain constraints, and to divide the implementation into fine-grained tasks, to generate more complete, readable, and maintainable code, as well as the need to continuously verify the code generated [36, 43, 48, 70, 75]. We extended these studies by systematically exploring the granularity and specificity of requirements and design documents necessary for successful code generation on a substantially larger end-to-end application and systematically evaluating the quality of the generated code.

The Software Engineering community at large focuses on developing GenAI tools to assist developers with various software-related tasks, such as requirements elicitation, architectural design, code and test generation, code understanding, code review, and debugging. Besides these individual tasks, several authors envisioned a paradigm shift in how developers control the software lifecycle, proposing, e.g., "requirement-driven end-user software engineering" [61], collaborative prompting [66], or multi-agent development systems [39, 68]. We omit discussing these approaches here as they are not directly relevant to our work on enhancing software engineering education.

To address a variety of issues in AI-generated code, developers anticipate code review to demand the same or even more time in the upcoming years, with AI tools becoming active participants in the code review process [33, 34]. Several AI-based automated code review tools have been recently proposed [38, 51, 57, 67, 71, 74]. Yet, evaluations of many such tools are typically conducted on benchmarks that oversimplify the real-world complexities; larger-scale evaluations [31, 76] show that state-of-the-art automated code review techniques still perform poorly in real-world code review scenarios, indicating the need for the increased developers' (and students') attention to code review, as well as to developing accurate and reliable code review tools.

8 Conclusion

As GenAI tools are reshaping the Software Engineering landscape, it becomes essential to identify both the skills necessary for effective practice and approaches to teaching these skills. By exploring students' experience in an upper-level undergraduate course on Software Engineering, as well as conducting our own systematic exploratory study developing a large-scale project of a realistic size and complexity (with an Android-based mobile client written in Kotlin and a Node.js-based cloud server written in TypeScript), our work takes a step in this direction. Our analysis shows that GenAI tools are not rendering traditional Software Engineering education obsolete. Instead, they further emphasize the need to develop the fundamental Software Engineering skills necessary to guide and evaluate AI assistants effectively.

Acknowledgments

This research was undertaken, in part, thanks to funding from the Canada Research Chairs Program.

9 Data Availability

To preserve students' private information, we only share aggregated data from the students' projects. We make our project assessment criteria, as well as the full description of the project we used for our exploratory study, all our prompts, the development strategies that we followed, the generated code, and our assessment of it publicly available to encourage reuse of our methodology and replication of our study [72].

References

[1] 2025. 2025 Stack Overflow Developer Survey. https://survey.stackoverflow.co/2025.
[2] 2026. Android Cursor Rule. https://cursor.directory/android.
[3] 2026. Android Studio. https://developer.android.com/studio.
[4] 2026. Authenticate Users With Sign In With Google. https://developer.android.com/identity/sign-in/credential-manager-siwg.
[5] 2026. Codacy. https://www.codacy.com.
[6] 2026. Cursor. https://cursor.com/get-started.
[7] 2026. Cursor Directory. https://cursor.directory.
[8] 2026. Cursor: Selecting Models. https://docs.cursor.com/en/guides/selecting-models.
[9] 2026. Espresso. https://developer.android.com/training/testing/espresso.
[10] 2026. Firebase Cloud Messaging. https://firebase.google.com/products/cloud-messaging.
[11] 2026. GitHub Copilot. https://github.com/features/copilot.
[12] 2026. Introducing Claude 4. https://www.anthropic.com/news/claude-4.
[13] 2026. ISO/IEC 25010:2023. https://www.iso.org/standard/78176.html.
[14] 2026. Jest. https://jestjs.io.
[15] 2026. Payload CMS Next.js TypeScript Best Practices. https://cursor.directory/payload-cms-nextjs-typescript-best-practices.
[16] 2026. Socket.IO. https://socket.io.
[17] 2026. The Movie Database (TMDB). https://developer.themoviedb.org.
[18] 2026. Top IDE Index. https://pypl.github.io/IDE.html.
[19] 2026. TypeScript Development Guidelines & Shortcuts. https://cursor.directory/typescript-development-guidelines-shortcuts.
[20] 2026. Vibe Coding. https://en.wikipedia.org/wiki/Vibe_coding.
[21] 2026. Visual Studio Code. https://code.visualstudio.com.
[22] 2026. Vue.js TypeScript Best Practices. https://cursor.directory/vuejs-typescript-best-practices.
[23] Altaf Allah Abbassi, Leuson Da Silva, Amin Nikanjam, and Foutse Khomh. 2025. A Taxonomy of Inefficiencies in LLM-Generated Code. In *Proceedings of the 41th International Conference on Software Maintenance and Evolution*. 1–12. doi:10.1109/ICSME64153.2025.00043
[24] Markus Aleksy, Tobias Hildenbrand, Claudia Obergfell, Martin Schader, and Michael Schwind. 2009. A Pragmatic Approach to Traceability in Model-Driven Development. In *PRIMIUM - Process Innovation for Enterprise Software*. Gesellschaft für Informatik e.V., 113–127.
[25] Isaac Alpizar-Chacon and Hieke Keuning. 2025. Student's Use of Generative AI as a Support Tool in an Advanced Web Development Course. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1*. 312–318. doi:10.1145/3724363.3729106
[26] Owura Asare, Meiyappan Nagappan, and N. Asokan. 2023. Is GitHub's Copilot as Bad as Humans at Introducing Vulnerabilities in Code? *Empirical Software Engineering* 28, 6 (2023), 1–24. doi:10.1007/S10664-023-10380-1
[27] Sahar Badihi, Michael Tegegn, Evelien Riddell, Krzysztof Czarnecki, and Julia Rubin. 2026. Hey, ChatGPT, Look at My Work: Using Conversational AI in Requirements Engineering.
[28] Joel Becker, Nate Rush, Beth Barnes, and David Rein. 2025. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/.
[29] Rudrajit Choudhuri, Dylan Liu, Igor Steinmacher, Marco Gerosa, and Anita Sarma. 2024. How Far Are We? The Triumphs and Trials of Generative AI in Learning Software Engineering. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13. doi:10.1145/3597503.3639201
[30] Rudrajit Choudhuri, Ambareesh Ramakrishnan, Amreeta Chatterjee, Bianca Trinkenreich, Igor Steinmacher, Marco Gerosa, and Anita Sarma. 2025. Insights from the Frontline: GenAI Utilization Among Software Engineering Students. In *Proceedings of the IEEE/ACM 37th International Conference on Software Engineering Education and Training*. 1–12. doi:10.1109/CSEET66350.2025.00007

[31] Umut Cihan, Vahid Haratian, Arda İçöz, Mert Kaan Gül, Ömercan Devran, Emircan Furkan Bayendur, Baykal Mehmet Uçar, and Eray Tüzün. 2025. Automated Code Review in Practice. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice*. 425–436. doi:10.1109/ICSE-SEIP66354.2025.00043

[32] Mike Cohn. 2004. *User Stories Applied: For Agile Software Development.* Addison Wesley Longman Publishing Co., Inc.

[33] Antonio Collante, Samuel Abedu, SayedHassan Khatoonabadi, Ahmad Abdellatif, Ebube Alor, and Emad Shihab. 2025. The Impact of Large Language Models (LLMs) on Code Review Process. https://arxiv.org/abs/2508.11034.

[34] Michael Dorner, Andreas Bauer, Darja Šmite, Lukas Thode, Daniel Mendez, Ricardo Britto, Stephan Lukasczyk, Ehsan Zabardast, and Michael Kormann. 2025. Quo Vadis, Code Review? Exploring the Future of Code Review. https://arxiv.org/abs/2508.06879.

[35] Yujia Fu, Peng Liang, Amjed Tahir, Zengyang Li, Mojtaba Shahin, Jiaxin Yu, and Jinfu Chen. 2025. Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study. *ACM Transactions on Software Engineering and Methodology* (2025), 1–34. doi:10.1145/3716848

[36] Simone Keller Fuchter, Mario Sergio Schlichting, and Geraldo Gurgel Filho. 2025. Study on the Use of ChatGPT for Generating Code for Web-Based Augmented Reality Applications. *Measurement: Sensors* 38 (2025), 101701. doi:10.1016/j.measen.2024.101701

[37] Simson Garfinkel, Mohan Sankaran, Rohan Sharma, Arunachalam Balasubramanian Shrinivass, Arpan Pandey, and Aruun Kumar. 2026. ACM TechBrief: AI-Assisted Software Development, or Vibe Coding: Benefits and Risks of AI-Driven Software Development. doi:10.1145/3807517.380

[38] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the Potential of ChatGPT in Automated Code Refinement: An Empirical Study. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13. doi:10.1145/3597503.3623306

[39] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30. doi:10.1145/3712003

[40] Huimin Hu, Yingying Wang, Julia Rubin, and Michael Pradel. 2025. An Empirical Study of Suppressed Static Analysis Warnings. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 290–311. doi:10.1145/3715729

[41] Kevin Jesse, Toufique Ahmed, Premkumar T. Devanbu, and Emily Morgan. 2023. Large Language Models and Simple, Stupid Bugs. In *Proceedings of the 20th International Conference on Mining Software Repositories*. 563–575. doi:10.1109/MSR59073.2023.00082

[42] Ahmed Kharrufa, Sami Alghamdi, Abeer Aziz, and Christopher Bull. 2025. LLMs Integration in Software Engineering Team Projects: Roles, Impact, and a Pedagogical Design Space for AI Tools in Computing Education. *ACM Transactions on Computing Education* (2025), 1–24. doi:10.48550/ARXIV.2410.23069

[43] Dae-Kyoo Kim. 2024. *Comparing Proficiency of ChatGPT and Bard in Software Development.* 25–51. doi:10.1007/978-3-031-55642-5_2

[44] Craig Larman. 2004. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development (3rd Edition).* Prentice Hall PTR.

[45] Hao-Ping (Hank) Lee, Advait Sarkar, Lev Tankelevitch, Ian Drosos, Sean Rintel, Richard Banks, and Nicholas Wilson. 2025. The Impact of Generative AI on Critical Thinking: Self-Reported Reductions in Cognitive Effort and Confidence Effects From a Survey of Knowledge Workers. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–22. doi:10.1145/3706598.3713778

[46] Shuang Li, Yuntao Cheng, Jinfu Chen, Jifeng Xuan, Sen He, and Weiyi Shang. 2024. Assessing the Performance of AI-Generated Code: A Case Study on GitHub Copilot. In *Proceedings of the IEEE 35th International Symposium on Software Reliability Engineering*. 216–227. doi:10.1109/ISSRE62328.2024.00030

[47] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*. 21558–21572.

[48] Mingxing Liu, Junfeng Wang, Tao Lin, Quan Ma, Zhiyang Fang, and Yanqun Wu. 2024. An Empirical Study of the Code Generation of Safety-Critical Software Using LLMs. *Applied Sciences* 14, 3 (2024), 1–41. doi:10.3390/app14031046

[49] Yue Liu, Thanh Le-Cong, Ratnadira Widyasari, Chakkrit Tantithamthavorn, Li Li, Xuan-Bach D. Le, and David Lo. 2024. Refining ChatGPT-Generated Code: Characterizing and Mitigating Code Quality Issues. *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024), 1–26. doi:10.1145/3643674

[50] Zhijie Liu, Yutian Tang, Xiapu Luo, Yuming Zhou, and Liang Feng Zhang. 2024. No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT. *IEEE Transactions on Software Engineering* 50, 6 (2024), 1548–1584. doi:10.1109/TSE.2024.3392499

[51] Junyi Lu, Lei Yu, Xiaojia Li, Li Yang, and Chun Zuo. 2023. LLaMA-Reviewer: Advancing Code Review Automation with Large Language Models through Parameter-Efficient Fine-Tuning. In *Proceedings of the IEEE 34th International Symposium on Software Reliability Engineering*. 647–658. doi:10.1109/ISSRE59848.2023.00026

[52] H. B. Mann and D. R. Whitney. 1947. On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. *The Annals of Mathematical Statistics* 18, 1 (1947), 50–60.

[53] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Zhen Ming (Jack) Jiang. 2023. GitHub Copilot AI Pair Programmer: Asset or Liability? *Journal of Systems and Software* 203, C (2023), 1–23. doi:10.1016/J.JSS.2023.111734

[54] Nhan Nguyen and Sarah Nadi. 2022. An Empirical Evaluation of GitHub Copilot's Code Suggestions. In *Proceedings of the 19th International Conference on Mining Software Repositories*. 1–5. doi:10.1145/3524842.3528470

[55] Steve R. Palmer and Mac Felsing. 2001. *A Practical Guide to Feature-Driven Development*. Pearson Education.

[56] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In *IEEE Symposium on Security and Privacy*. 754–768. doi:10.1145/3610721

[57] Chanathip Pornprasit and Chakkrit Tantithamthavorn. 2024. Fine-Tuning and Prompt Engineering for Large Language Models-Based Code Review Automation. *Information and Software Technology* 175, C (2024), 1–12. doi:10.1016/j.infsof.2024.107523

[58] Nishat Raihan, Mohammed Latif Siddiq, Joanna C.S. Santos, and Marcos Zampieri. 2025. Large Language Models in Computer Science Education: A Systematic Literature Review. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1*. 938–944. doi:10.1145/3641554.3701863

[59] Sanka Rasnayaka, Guanlin Wang, Ridwan Shariffdeen, and Ganesh Neelakanta Iyer. 2024. An Empirical Study on Usage and Perceptions of LLMs in a Software Engineering Project. In *Proceedings of the 1st International Workshop on Large Language Models for Code*. 111–118. doi:10.1145/3643795.3648379

[60] Mark Richards. 2015. *Software Architecture Patterns*. O'Reilly Media, Inc.

[61] Diana Robinson, Christian Cabrera, Andrew D. Gordon, Neil D. Lawrence, and Lars Mennen. 2025. Requirements Are All You Need: The Final Frontier for End-User Software Engineering. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–22. doi:10.1145/3708524

[62] Marie Salomon, Kyle D. Chin, Reid Holmes, Thomas Fritz, and Gail C. Murphy. 2025. An Exploration of How Generative AI Affects Workflow and Collaboration in a Software Engineering Course. In *Proceedings of the 2025 ACM SIGPLAN International Symposium on SPLASH-E*. 42–54. doi:10.5281/ZENODO.16782575

[63] Anshul Shah, Anya Chernova, Elena Tomson, Leo Porter, William G. Griswold, and Adalbert Gerald Soosai Raj. 2025. Students' Use of GitHub Copilot for Working with Large Code Bases. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1*. 1050–1056. doi:10.1145/3641554.3701800

[64] Charles Spearman. 1904. The Proof and Measurement of Association between Two Things. *The American Journal of Psychology* 15, 1 (1904), 72–101.

[65] Anselm Strauss and Juliet Corbin. 1998. *Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory*. Thousand Oaks, CA: Sage.

[66] Hari Subramonyam, Divy Thakkar, Andrew Ku, Juergen Dieber, and Anoop K. Sinha. 2025. Prototyping with Prompts: Emerging Approaches and Challenges in Generative AI Design for Collaborative Software Teams. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–22. doi:10.1145/3706598.3713166

[67] Xunzhu Tang, Kisub Kim, Yewei Song, Cedric Lothritz, Bei Li, Saad Ezzini, Haoye Tian, Jacques Klein, and Tegawendé F. Bissyandé. 2024. CodeAgent: Autonomous Communicative Agents for Code Review. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 11279–11313. doi:10.18653/v1/2024.emnlp-main.632

[68] Valerio Terragni, Annie Vella, Partha Roop, and Kelly Blincoe. 2025. The Future of AI-Driven Software Engineering. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–20 pages. doi:10.1145/3715003

[69] Catherine Tony, Nicolás E. Díaz Ferreyra, Markus Mutas, Salem Dhif, and Riccardo Scandariato. 2025. Prompting Techniques for Secure Code Generation: A Systematic Investigation. *ACM Transactions on Software Engineering and Methodology* (2025), 1–51. doi:10.1145/3722108

[70] Jonathan Ullrich, Matthias Koch, and Andreas Vogelsang. 2025. From Requirements to Code: Understanding Developer Practices in LLM-Assisted Software Engineering. In *Proceedings of the 33rd IEEE International Requirements Engineering Conference*. 1–10. doi:10.1109/RE63999.2025.00032

[71] Manushree Vijayvergiya, Małgorzata Salawa, Ivan Budiselić, Dan Zheng, Pascal Lamblin, Marko Ivanković, Juanjo Carin, Mateusz Lewko, Jovan Andonov, Goran Petrović, Daniel Tarlow, Petros Maniatis, and René Just. 2024. AI-Assisted Assessment of Coding Practices in Modern Code Review. In *Proceedings of the 1st ACM International Conference on AI-Powered Software*. 85–93. doi:10.1145/3664646.3665664

[72] Yingying Wang, Masih Beigi Rizi, Fatemeh Khashei, and Julia Rubin. 2026. Online Appendix. https://resess.github.io/artifacts/GenAIExperience/.

[73] Zhijie Wang, Zijie Zhou, Da Song, Yuheng Huang, Shengmai Chen, Lei Ma, and Tianyi Zhang. 2025. Towards Understanding the Characteristics of Code Generation Errors Made by Large Language Models. In *Proceedings of IEEE/ACM 47th International Conference on Software Engineering*. 2587–2599. doi:10.1109/ICSE55347.2025.00180

[74] Yongda Yu, Guoping Rong, Haifeng Shen, He Zhang, Dong Shao, Min Wang, Zhao Wei, Yong Xu, and Juhong Wang. 2024. Fine-Tuning Large Language Models to Improve Accuracy and Comprehensibility of Automated Code Review. *ACM Transactions on Software Engineering and Methodology* 34, 1 (2024), 1–26. doi:10.1145/3695993

[75] Edeline Priscilla Yusuf and Eddy Yusuf. 2025. Human-ChatGPT Co-created Simulations: Drop-off Zone Kinematics Case Study. *LEARNING:Jurnal Inovasi Penelitian Pendidikan dan Pembelajaran* 5, 2 (2025), 1–10. doi:10.51878/learning.v5i2.5352

[76] Zhengran Zeng, Ruikai Shi, Keke Han, Yixin Li, Kaicheng Sun, Yidong Wang, Zhuohao Yu, Rui Xie, Wei Ye, and Shikun Zhang. 2025. Benchmarking and Studying the LLM-based Code Review. https://arxiv.org/abs/2509.01494.

Received 2025-09-12; accepted 2025-12-22